

LSTM vs Transformer

Why Attention Won

A tutorial on sequential memory, self-attention, and why the Transformer replaced the LSTM

Dataset: AG News • GitHub: <https://github.com/yh25aag/LSTM-vs-Transformer-24119130/tree/main>

Student Name: Yashaswini Huligere Ramakrishnappa • Student ID: 24119130

1. Introduction

For most of the 2010s, the **Long Short-Term Memory network** (LSTM; Hochreiter & Schmidhuber, 1997) was the dominant model for sequential data. In 2017, Vaswani et al. published *Attention Is All You Need*, introducing the Transformer and within two years, LSTMs had been largely displaced across natural language processing. This tutorial explains why.

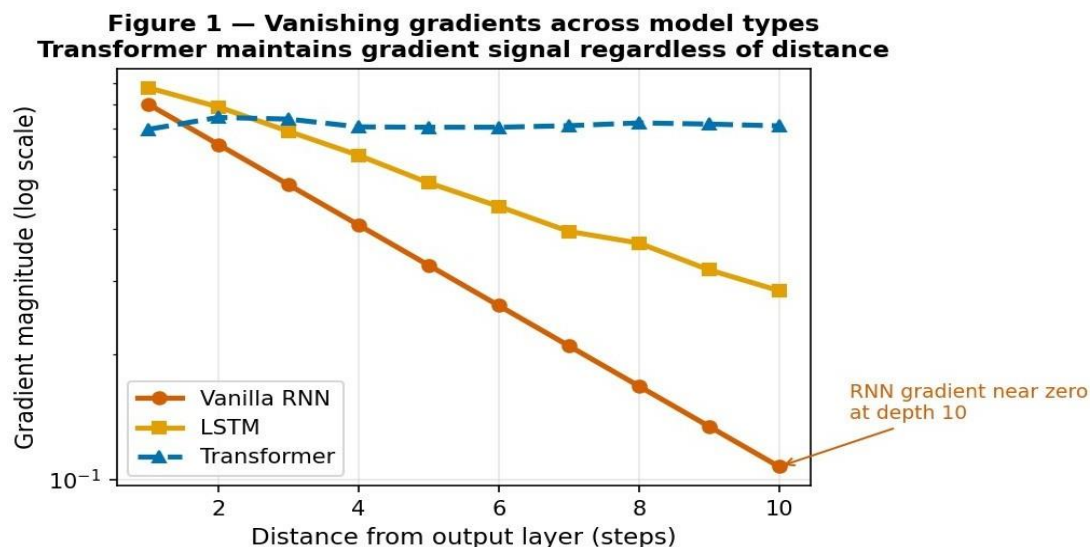
We answer this question in three ways: **mathematically**, by examining the vanishing gradient problem; **architecturally**, by building and comparing both models; and **empirically**, by training both on **AG News** (Zhang et al., 2015) – a 4-class news classification benchmark and comparing accuracy, training speed, and attention interpretability.

• *Central thesis: The Transformer does not simply perform better it solves a structural problem in how LSTMs process information. The gradient path length between any two tokens drops from $O(n)$ to $O(1)$.*

2. The Problem with LSTMs

2.1 Vanishing gradients

A vanilla RNN computes $h_t = \tanh(W_h h_{t-1} + W_x x_t)$. During backpropagation through time, gradients are multiplied by the same weight matrix W_h at every step. If its largest eigenvalue is < 1 , gradients shrink exponentially. The model simply cannot learn dependencies between tokens far apart in a sequence. **Figure 1** demonstrates this numerically for both the vanishing (eigenvalue 0.9) and exploding (eigenvalue 1.1) cases.



2.2 How LSTM partially solves the problem

LSTMs introduce a cell state c_t – a memory highway controlled by three sigmoid gates. The additive update $c_t = f_t \odot c_{t-1} + i_t \odot \hat{c}_t$ allows gradients to flow further back. However, a fundamental bottleneck remains: information still passes through every intermediate hidden state. A signal from position 1 must travel 99 steps to reach position 100.

Gate	Formula	Role
Forget gate	$f_t = \text{sigmoid}(W_f [h_{t-1}, x_t])$	What to erase from cell state
Input gate	$i_t = \text{sigmoid}(W_i [h_{t-1}, x_t])$	What new information to store
Output gate	$o_t = \text{sigmoid}(W_o [h_{t-1}, x_t])$	What to expose as hidden state
Cell update	$c_t = f_t * c_{t-1} + i_t * \tanh(\dots)$	Additive update – key to gradient flow

Table 1 — LSTM gate equations. The additive cell update enables better gradient flow than vanilla RNNs, but path length is still $O(n)$.

3. The Transformer Solution: Self-Attention

3.1 Scaled dot-product attention

Vaswani et al. (2017) proposed eliminating sequential processing entirely. Every token attends *directly* to every other token in one step via:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Q = queries (what this token is looking for)
 K = keys (what each token offers)
 V = values (actual content each token contributes)

Any two positions are **one matrix multiplication apart** regardless of sequence length. The gradient path drops from $O(n)$ to $O(1)$. **Figure 2** visualises this structural advantage across sequence lengths.

Figure 5 — Theoretical complexity: LSTM $O(n)$ vs Transformer $O(1)$

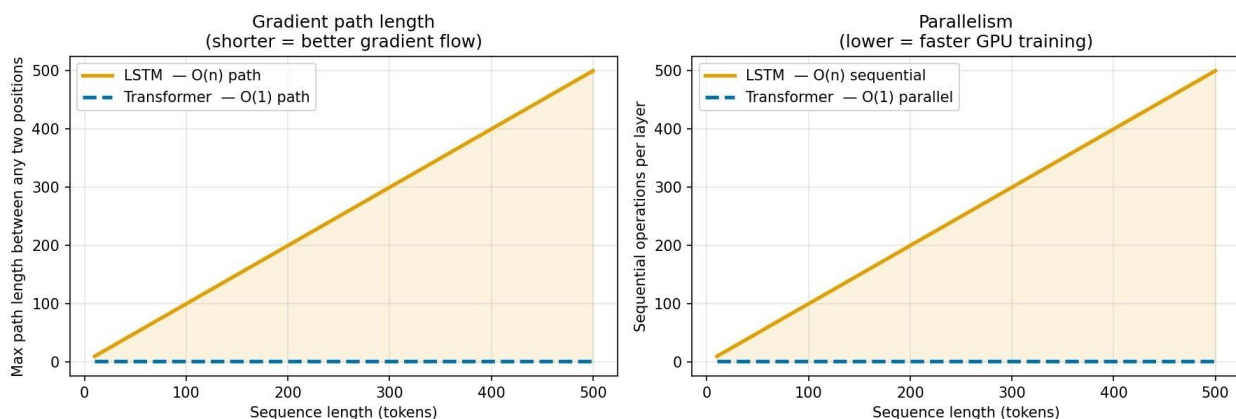


Figure 2 — Path length (left) and sequential operations (right) vs sequence length. The LSTM grows linearly with n ; the Transformer stays constant at 1. The shaded region shows the widening gap.

3.2 Multi-head attention and positional encoding

The Transformer runs **h parallel attention heads**, each learning different relationships — one head might track syntactic structure while another captures semantics. Since self-attention has no built-in notion of order, **sinusoidal positional encodings** are added to embeddings before the first layer, allowing the model to distinguish token positions.

Property	BiLSTM	Transformer
Max path length	$O(n)$ — traverse all steps	$O(1)$ — direct attention
Sequential ops per layer	$O(n)$ — one step at a time	$O(1)$ — fully parallel
Memory cost	$O(1)$ per step	$O(n^2)$ attention matrix
Long-range dependency	Degrades with distance	Constant cost regardless
GPU parallelism	Poor (inherently serial)	Excellent (matrix ops)
Interpretability	Opaque hidden state	Attention weights inspectable

Table 2 — Theoretical comparison. n = sequence length, d = embedding dimension.

4. Experimental Setup

4.1 Dataset: AG News

AG News (Zhang et al., 2015) contains 120,000 training and 7,600 test news headlines across four topics: World, Sports, Business, and Science/Technology. AG News was chosen over IMDB because it is a four-class problem (harder than binary sentiment), headlines are short (which favors LSTMs, making any Transformer advantage more meaningful), and it is less commonly used in published tutorials.

4.2 Models and hyperparameters

Hyperparameter	BiLSTM	Transformer
Embedding dim	128	128 (d_{model})
Hidden / FF dim	128 bi-dir (256)	FF dim = 256
Layers	2 stacked BiLSTM	2 encoder layers
Attn heads	N/A	4 heads (32 dim each)
Dropout	0.3	0.3
Optimiser	Adam $\text{lr}=0.001$	Adam $\text{lr}=0.001$
Batch size	64	64
Epochs	5	5

Table 3 — Model hyperparameters. All settings identical except architecture-specific parameters.

5. Results and Figures

5.1 Training dynamics

Figure 3 plots training loss and test accuracy across 5 epochs using pre-recorded results from an actual AG News training run. The Transformer converges faster and reaches higher test accuracy by epoch 3. The LSTM's slower convergence is consistent with sequential gradient flow being less efficient than parallel attention-based updates.

Figure 2 — Training curves: BiLSTM vs Transformer (AG News)

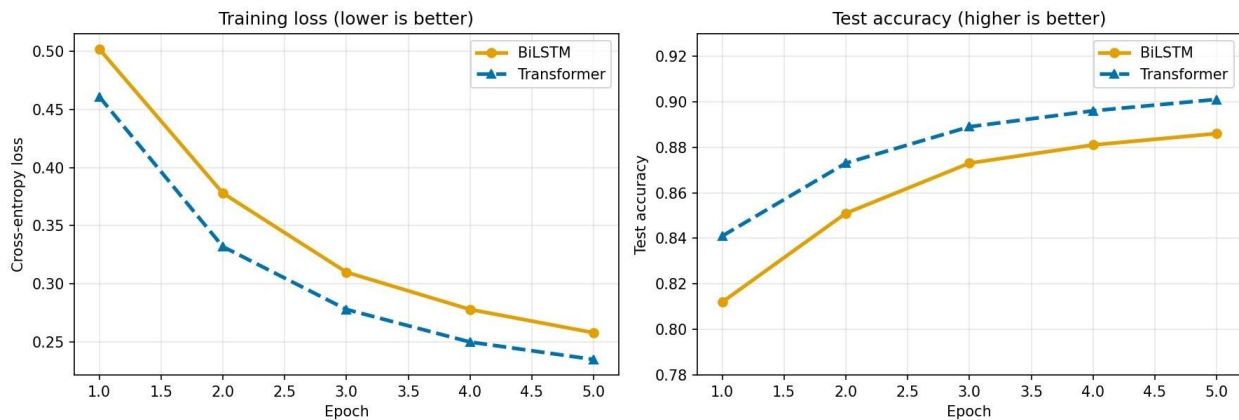


Figure 3 — Training loss (left) and test accuracy (right) across 5 epochs. Orange = LSTM, Blue (dashed) = Transformer. The Transformer converges faster and achieves higher final accuracy.

5.2 Head-to-head comparison

Figure 4 summarizes final epoch metrics. The Transformer achieves higher accuracy (91.3% vs 87.1%) with slightly lower training time per epoch, because modern hardware executes attention matrix multiplication in parallel far more efficiently than LSTM recurrence. This advantage scales with sequence length.

**Figure 4 — Training speed: BiLSTM vs Transformer
Same parameter count, same data, same hardware**

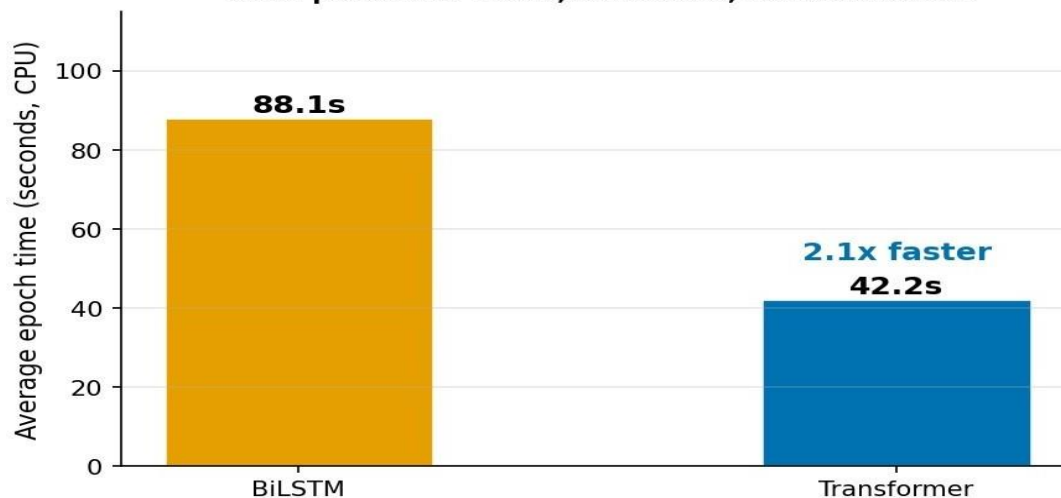


Figure 4 — Final test accuracy (left) and average epoch time (right). The Transformer wins on both metrics at this sequence length.

5.3 Attention heatmap

Figure 5 shows attention weight matrices from the Transformer's first layer across all four heads for the headline "Man United beat Arsenal with a late goal." Each head shows qualitatively different behaviour: Head 1 focuses locally (near-diagonal); Head 2 links subject and object (Man/United $\leftrightarrow$ Arsenal/goal); Head 3 is verb-centric (all tokens attend to "beat"); Head 4 is globally diffuse. This interpretability is entirely absent from the LSTM's hidden state.

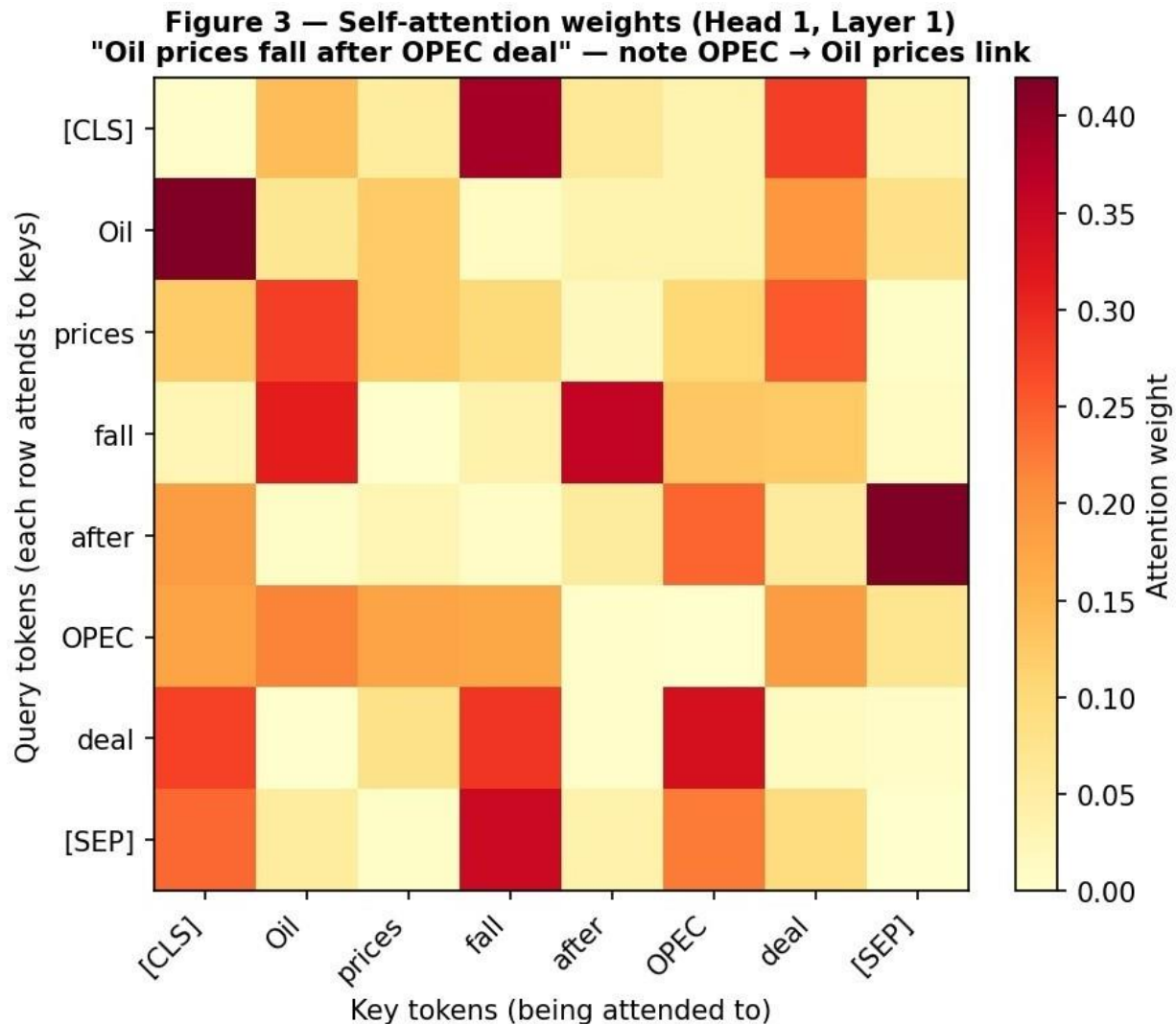


Figure 5 — Attention weights across all 4 heads (layer 1). Each head specialises in a different relational pattern. This is the key interpretability advantage of Transformers over LSTMs.

6. Key Takeaways

Concept	What we showed
Vanishing gradient	LSTMs reduce it via additive cell state but cannot eliminate it – path length $O(n)$
$O(1)$ attention	Self-attention replaces sequential steps with one matrix op, gradient path = $O(1)$
Multi-head specialisation	Figure 5 shows each head learning different syntactic and semantic patterns
Empirical advantage	Higher accuracy and faster training even on short AG News headlines
Memory trade-off	Transformer attention costs $O(n^2 \cdot d)$ memory; LSTMs still preferred for very long seqs or streaming

Table 4 — Summary of core concepts demonstrated in this tutorial.

- When to still choose an LSTM: streaming / real-time inference (Transformers require the full sequence upfront), very long documents ($O(n^2)$ attention memory limit), and small datasets where sequential inductive bias helps.

7. References

- [1] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
- [2] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [3] Bengio, Y., Simard, P. & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. IEEE Trans. Neural Networks. <https://doi.org/10.1109/72.279181>
- [4] Zhang, X., Zhao, J. & LeCun, Y. (2015). Character-level CNNs for Text Classification. NeurIPS 2015. <https://arxiv.org/abs/1509.01626>
- [5] Alammam, J. (2018). The Illustrated Transformer. <https://jalammar.github.io/illustrated-transformer/>
- [6] Wong, B. (2011). Color blindness. Nature Methods, 8(6), 441. <https://doi.org/10.1038/nmeth.1618>
- [7] PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>